

Document Summary

This note provides an in-depth overview of Model-Free Reinforcement Learning, focusing on prediction and control methods. It covers Monte-Carlo methods, Temporal-Difference learning, and various control algorithms like SARSA and Q-Learning. The note also discusses the differences between on-policy and off-policy learning, the importance of exploration, and the use of eligibility traces in $TD(\lambda)$.

Model-Free Reinforcement Learning: Prediction and Control

Introduction

- **Model-Free Reinforcement Learning:** Learning directly from episodes of experience without knowledge of the MDP transition/rewards.
- **Model-Free Prediction:** Estimate the value function of an unknown MDP.
- **Model-Free Control:** Optimize the value function of an unknown MDP.

Monte-Carlo Methods

- **Monte-Carlo (MC) Reinforcement Learning:**
 - MC methods learn directly from episodes of experience.
 - MC is model-free: no knowledge of MDP transitions/rewards.
 - MC learns from complete episodes: no bootstrapping.
 - MC uses the simplest possible idea: value = mean return across episodes.
 - Limitation: can only apply MC to episodic MDPs. All episodes must terminate.
- **Monte-Carlo Policy Evaluation:**
 - Goal: learn v^π from episodes of experience under policy π .
 - Return is the total discounted reward.
 - Value function is the expected return $v^\pi(s) = \mathbb{E}[G_t | S_t = s]$.
 - Monte-Carlo policy evaluation uses empirical mean return instead of expected return.
 - $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$.
- **First-Visit Monte-Carlo Policy Evaluation:**

- To evaluate state s :
 - The first time step t that state s is visited in an episode.
 - Increment counter $N_s \leftarrow N_s + 1$.
 - Increment total return $S_s \leftarrow S_s + G_t$.
 - Value is estimated by mean return $V_s = S(s)/N_s$.
 - Update at the end of the episode.
 - By law of large numbers, $V_s \rightarrow v^\pi(s)$ as $N_s \rightarrow \infty$.
- **Every-Visit Monte-Carlo Policy Evaluation:**
 - To evaluate state s :
 - Every time step t that state s is visited in an episode.
 - Increment counter $N_s \leftarrow N_s + 1$.
 - Increment total return $S_s \leftarrow S_s + G_t$.
 - Value is estimated by mean return $V_s = S(s)/N_s$.
- **Blackjack Example:**
 - States (200 of them):
 - Current sum (12-21)
 - Dealer's showing card (ace-10)
 - Do I have a useable ace? (yes-no)
 - Reward for action stick (Stop receiving cards (and terminate)):
 - +1 if sum of cards > sum of dealer cards
 - 0 if sum of cards = sum of dealer cards
 - -1 if sum of cards < sum of dealer cards
 - Reward for action twist (Take another card (no replacement)):
 - -1 if sum of cards > 21 (and terminate)
 - 0 otherwise
 - Transitions: automatically twist if sum of cards < 12
- **Blackjack Value Function after MC Learning:**
 - Usable ace
 - No useable ace
 - After 10K episodes
 - After 500K episodes
 - Policy: stick if sum of cards ≥ 20 , otherwise twist
- **Incremental Mean:**

- The mean $\mu_1, \mu_2, \dots$ of a sequence $x_1, x_2, \dots$ can be computed incrementally:
 - $\mu_k = \frac{1}{k}x_k + \frac{k-1}{k}\mu_{k-1} = \mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})$
- **Incremental Mean MC Update:**
 - Update $V(s)$ incrementally after episode $S_1, A_1, R_1, \dots, R_T$.
 - For each state S_t with return G_t :
 - Increment counter $N(s) \leftarrow N(s) + 1$.
 - Update value function (with incremental mean):
 - $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S)}(G - V(S))$.
 - In non-stationary problems, track a running mean (forget old episodes):
 - $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$.

Temporal-Difference Learning

- **Temporal-Difference (TD) Learning:**
 - TD methods learn directly from episodes of experience.
 - TD is model-free: no knowledge of MDP transitions/rewards.
 - TD learns from incomplete episodes, by bootstrapping.
 - While MC learns from complete ones.
 - TD updates a guess towards a guess.
- **Goal:** learn v^π from episodes of experience under policy π .
- **Incremental every-visit MC.**
- **Update value $V(S_t)$ toward actual return G_t :**
 - $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$.
- **Simplest temporal-difference learning algorithm (TD(0)):**
 - Update value $V(S_t)$ toward estimated return $R_{t+1} + \gamma V(S_{t+1})$:
 - $V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$.
- **MC vs TD Learning:**
 - TD error δ_t .
 - TD target.
- **Driving Home Example:**
 - Predicted total travel time.
- **Advantages and Disadvantages of MC vs. TD (I):**
 - TD can learn before knowing the final outcome.
 - TD can learn online after every step.
 - MC must wait until end of episode before return is known.

- TD can learn without the final outcome.
- TD can learn from incomplete sequences.
- MC can only learn from complete sequences.
- TD works in continuing (non-terminating) environments.
- MC only works for episodic (terminating) environments.
- **Bias-Variance Tradeoff:**
 - Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is unbiased estimate of $v^\pi(S_t)$.
 - True TD target $R_{t+1} + \gamma v^\pi(S_{t+1})$ is unbiased estimate of $v^\pi(S_t)$.
 - TD target $R_{t+1} + \gamma V(S_{t+1})$ is biased estimate of $v^\pi(S_t)$.
 - TD target is much lower variance than the return:
 - Return depends on many random actions, transitions, rewards.
 - TD target depends on one random action, transition, reward.
- **Advantages and Disadvantages of MC vs. TD (II):**
 - MC has high variance, zero bias.
 - Good convergence properties (even with function approximation).
 - Not very sensitive to initial value.
 - Very simple to understand and use.
 - TD has low variance, some bias.
 - Usually more efficient than MC.
 - TD(0) converges to $v^\pi(s)$ (but not always with function approximation).
 - More sensitive to initial value.
- **Random Walk Example:**
 - Estimated value.
- **Random Walk Example – MC vs TD:**
 - RMS error (averaged over states).
- **Batch MC and TD:**
 - MC and TD converge: $V(s) \rightarrow v^\pi(s)$ as experience $\rightarrow \infty$.
 - But what about batch solution for finite experience?
 - e.g. repeated sample episode $k \in [1, K]$.
 - Apply MC or TD(0) to episode k :
 - $s_1^1, a_1^1, r_2^1, \dots, s_T^1$.
 - $\vdots$.
 - $s_1^K, a_1^K, r_2^K, \dots, s_T^K$.

- **A Simple Example:**
 - Two states A; B; no discounting; 8 episodes of experience.
 - 1. A, 0, B, 0
 - 2. B, 1
 - 3. B, 1
 - 4. B, 1
 - 5. B, 1
 - 6. B, 1
 - 7. B, 1
 - 8. B, 0
 - What is $V(A)$; $V(B)$?
- **Certainty Equivariance:**
 - MC converges to solution with minimum mean-squared error.
 - Best fit to the observed returns.
 - TD(0) converges to solution of maximum likelihood Markov model.
 - Solution to the MDP $\mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma$ that best fits the data.
- **Advantages and Disadvantages of MC vs. TD (III):**
 - TD exploits Markov property.
 - Usually more efficient in Markov environments.
 - MC does not exploit Markov property.
 - Usually more effective in non-Markov environments.
- **Unified View:**
 - MC Update: $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$.
 - TD Update: $V(S_t) \leftarrow V(S_t) + \alpha(R_t + \gamma V(S_{t+1}) - V(S_t))$.
 - Dynamic Programming: $V(S_t) \leftarrow \mathbb{E}[R_t + 1 + \gamma V(S_{t+1})]$.
- **Bootstrapping and Sampling:**
 - Bootstrapping - Update involves an estimate.
 - MC does not bootstrap.
 - DP bootstraps.
 - TD bootstraps.
 - Sampling - Update samples an expectation.
 - MC samples.
 - DP does not sample.

- TD samples.
- **Unified View of RL:**

Generalizing TD

- **n-step Prediction:**
 - Have TD look and target n steps in the future $n \rightarrow \infty$.
- **n-step Return:**
 - Consider the following n -step returns for $n = 1, 2, \dots, \infty$.
 - Define the n -step return.
 - Learn based on the n -step difference.
- **Averaging n-step Returns:**
 - We can average n -step returns over different n .
 - E.g.: Average the 2-step and 4-step returns $\frac{1}{2}G^{(2)} + \frac{1}{2}G^{(4)}$.
 - Combines information from two different time-steps.
 - Can we efficiently combine information from all time-steps?
- **λ -returns:**
 - The λ -return G_t^λ combines all n -step returns $G_t^{(n)}$.
 - Using weight $(1 - \lambda)\lambda^{n-1}$.
 - Update as appropriate (TD(λ)):
 - $V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t))$.
 - $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$.
- **TD(λ) Weight Function:**
 - $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$.
- **Forward View TD(λ):**
 - Update value function towards the λ -return.
 - Forward-view looks into the future to compute G_t^λ .
 - Like MC, can only be computed from complete episodes.
- **Backward View TD(λ):**
 - Forward view provides theory.
 - Backward view provides mechanism.
 - Update online, every step, from incomplete sequences.
- **Eligibility Traces:**
 - Credit assignment problem: what caused shock?

- Frequency heuristic: assign credit to most frequent states.
- Recency heuristic: assign credit to most recent states.
- Eligibility traces combine both heuristics.
- $E_0(s) = 0$.
- $E_t(s) = \gamma\lambda E_{t-1}(s)$.
- Accumulate eligibility trace $+1(S_t; s)$ times of visit to state.
- **Backward View TD(λ):**
 - Keep an eligibility trace for every state s .
 - Update value $V(s)$ for every state s .
 - In proportion to TD-error δ_t and eligibility trace $E_t(s)$.
 - $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$.
 - $V(s) = V(s) + \alpha \delta E(s)$.
- **TD(λ) and TD(0):**
 - When $\lambda = 0$ only current state is updated.
 - $E_t(s) = 1(S_t; s)$.
 - $V(s) \leftarrow V(s) + \alpha \delta_t E_t(s)$.
 - Equivalent to TD(0) update.
 - $V(S_t) \leftarrow V(S_t) + \alpha \delta_t$.
- **TD(λ) and MC:**
 - When $\lambda = 1$ credit is deferred until end of episode.
 - Consider episodic environments with offline updates.
 - Over the course of an episode, total update for TD(1) is the same as total update for MC.
 - The sum of offline updates is identical for forward-view and backward-view TD(λ).
 - Theorem.
- **Telescoping in TD(1):**
 - When $\lambda = 1$ sum of TD errors telescopes into MC error.
 - TD(1) is roughly equivalent to every-visit Monte-Carlo.
 - Error is accumulated online, step-by-step.
 - If value function is only updated offline at end of episode, then total update is the same as MC.

Model-Free Control

- **Outline:**

- Introduction
- On-policy vs Off-policy
- On-policy Monte-Carlo
- On-policy Temporal-Difference learning (SARSA)
- Off-policy Temporal-Difference (Q-learning)
- **Introduction:**
 - Model-Free Control – Where to find it:
 - Elevator
 - Robot walking
 - Vehicle Steering
 - Bioreactor
 - Molecule engineering
 - Robocup Soccer
 - Quake
 - Portfolio management
 - Protein Folding
 - Game of Go
 - For most of these problems, either:
 - MDP model is unknown, but experience can be sampled.
 - MDP model is known, but is too big to use, except by samples.
 - Model-free control can solve these problems.
- **On-policy & Off-policy Learning:**
 - On-policy learning:
 - Learn on the job.
 - Learn about policy π from experience sampled from π .
 - Off-policy learning:
 - Look over someone's shoulder.
 - Learn about policy π from experience sampled from μ .
- **On-policy Monte-Carlo:**
- **Generalized Policy Iteration (Lecture 5b):**
 - Policy evaluation - Estimate v^π .
 - Any policy evaluation.
 - Policy improvement - Generate $\pi' \geq \pi$.

- Any policy improvement algorithm.
- **Generalized Policy Iteration with On-policy MC:**
 - Policy evaluation - Monte-Carlo policy evaluation, $V = v^\pi$?
 - Policy improvement - Generate greedy policy improvement?
- **Model-Free Policy Iteration Using Action-Value Function:**
 - Greedy policy improvement over $V(s)$ requires model of MDP.
 - Greedy policy improvement over $Q(s, a)$ is model-free:
 - $\pi'(s) = \arg \max_{a \in \mathcal{A}} R(s, a) + P(s, s', a)V(s')$.
 - $\pi'(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$.
- **Generalized Policy Iteration with Action-Value Function:**
 - Policy evaluation - Monte-Carlo policy evaluation, $Q = q^\pi$.
 - Policy improvement - Generate Greedy policy improvement?
- **Example of Greedy Action Selection:**
 - There are two doors in front of you.
 - You open the left door and get reward 0 – $V(\text{left}) = 0$.
 - You open the right door and get reward +1 – $V(\text{right}) = +1$.
 - You open the right door and get reward +3 – $V(\text{right}) = +2$.
 - You open the right door and get reward +2 – $V(\text{right}) = +2$.
 - ... Are you sure you've chosen the best door?
- **ϵ -greedy Exploration:**
 - Simplest idea for ensuring continual exploration.
 - All m actions are tried with non-zero probability.
 - With probability $1 - \epsilon$ choose the greedy action.
 - With probability ϵ choose an action at random.
- **ϵ -greedy Policy Improvement:**
 - For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q^π is an improvement $v^{\pi'}(s) \geq v^\pi(s)$.
 - Theorem.
 - Therefore from policy improvement theorem $v^{\pi'}(s) \geq v^\pi(s)$.
- **Monte-Carlo Policy Iteration:**
 - Policy evaluation - Monte-Carlo policy evaluation, $Q \approx q^\pi$.
 - Policy improvement - ϵ -greedy policy improvement.
- **Monte-Carlo Control:**

- Every Episode:
 - Policy evaluation - Monte-Carlo policy evaluation, $Q \approx q^\pi$.
 - Policy improvement - ϵ -greedy policy improvement.
- **Greedy in the Limit with Infinite Exploration (GLIE):**
 - ϵ -greedy is GLIE if ϵ reduces to zero at $\epsilon_k = \frac{1}{k}$.
 - The policy converges on a greedy policy.
 - Definition (Greedy in the Limit with Infinite Exploration - GLIE):
 - All state-action pairs are explored infinitely many times.
- **GLIE Monte Carlo Control:**
 - Sample k -th episode using π : $\{S_1, A_1, R_2, \dots, S_T\} \sim \pi$.
 - For each state S_t and action A_t in the episode:
 - Improve policy based on new action-value function.
 - GLIE Monte Carlo control converges to the optimal action-value function $Q(s, a) \rightarrow q^*(s, a)$.
 - Theorem.
 - $\epsilon \leftarrow \frac{1}{k}$.
 - $\pi \leftarrow \epsilon$ -greedy(Q).
- **On-Policy TD Control:**
- **MC Vs TD Control:**
 - TD learning has several advantages over MC:
 - Lower variance.
 - Online.
 - Incomplete sequences.
 - Straightforward intuition - Use TD instead of MC in our control loop.
 - Apply TD to $Q(s, a)$.
 - Use ϵ -greedy policy improvement.
 - Update every time-step.
- **Updating Action-Value Functions with SARSA:**
 - $Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$.
 - $S, A \ R \ S' \ A'$.
 - We sample also future action A' (instead of leveraging policy to compute expectation).
 - Expected SARSA.
- **On-Policy Control with SARSA:**

- Every time-step:
 - Policy evaluation - SARSA, $Q \approx q^\pi$.
 - Policy improvement - ϵ -greedy policy improvement.
- **SARSA Algorithm for On-Policy Control:**
- **Convergence of SARSA:**
 - SARSA converges to the optimal action-value function ($Q(s, a) \rightarrow q^*(s, a)$) under the following conditions:
 - GLIE sequence of policies $\pi_t(a|s)$.
 - Robbins-Monro sequence of step-sizes α_t .
 - Theorem.
- **Time for TD Demo:**
 - https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_td.html
- **SARSA(λ):**
- **n-step SARSA:**
 - Consider the following n-step returns for $n = 1, 2, \dots, \infty$.
 - Define the n-step Q-return.
 - n-step SARSA updates $Q(S, A)$ towards the n-step Q-return.
- **SARSA backups:**
- **SARSA(λ) - Forward View:**
 - The q^λ return combines all n-step Q-returns $q_t^{(n)}$.
 - Using weight $(1 - \lambda)\lambda^{n-1}$.
 - Forward SARSA update $Q(S, A) \leftarrow Q(S, A) + \alpha(q_t^\lambda - Q(S, A))$.
 - SARSA(λ).
- **SARSA(λ) - Backward View:**
 - The return of eligibility traces.
 - SARSA(λ) needs one eligibility trace for each state-action pair.
 - $E_0(s, a) = 0$.
 - $E_t(s, a) = \gamma\lambda E_{t-1}(s, a) + 1(S_t, A_t; s, a)$.
 - $Q(s, a)$ is updated for every state s and action a in proportion to TD-error δ_t and eligibility trace $E_t(s, a)$.
 - $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$.
 - $Q(s, a) \leftarrow Q(s, a) + \alpha\delta_t E_t(s, a)$.
- **SARSA(λ) Algorithm:**

- **SARSA(λ) on Gridworld:**
- **Off-policy TD Learning:**
- **Off-Policy Learning:**
 - Evaluate target policy $\pi(a|s)$ to compute $v^\pi(s)$ or $q^\pi(s)$.
 - While following behaviour policy $\mu(a|s)$.
 - $\{S_1, A_1, R_2, \dots, S_T\} \sim \mu$.
 - Why is this important?
 - Learn from imitation (humans, other agents, ...).
 - Re-use experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{t-1}$.
 - Learn about optimal policy while following exploratory policy.
 - Learn about multiple policies while following one policy.
- **Importance Sampling:**
 - Estimate the expectation leveraging an external (importance) distribution.
 - Draw samples from importance distribution $Q(X)$ rather than from $P(X)$.
 - Assign weights such that the empirical expectation (on $Q(X)$ samples) matches the expectation under $P(X)$.
- **Importance Sampling for Off-policy Monte Carlo:**
 - Use returns generated from μ to evaluate π .
 - Weight return G_t according to similarity between policies.
 - Multiply importance sampling corrections along whole episode.
 - Update value towards corrected return.
 - Importance sampling can dramatically increase variance.
- **Importance Sampling for Off-policy TD:**
 - Use TD targets generated from μ to evaluate π .
 - Weight TD targets $R + \gamma V(S')$ by importance sampling.
 - Only need a single importance sampling correction.
 - Much lower variance than MC.
 - Policies only need to be similar over a single step.
- **Q-Learning:**
 - Off-policy learning of action-values $Q(s, a)$.
 - No importance sampling is required.
 - Next action is chosen using behaviour policy $A_{t+1} \sim \mu(\cdot|S_t)$.
 - But we consider alternative successor action $A' \sim \pi(\cdot|S_t)$.

- And update $Q(S_t, A_t)$ towards value of alternative action:
 - $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t))$.
- **Off-policy Control by Q-Learning:**
 - Allow both behaviour and target policies to improve.
 - The target policy π is greedy w.r.t. $Q(S_t, A_t)$.
 - The behaviour policy μ is ϵ -greedy w.r.t. $Q(s, a)$.
 - The Q-learning target then simplifies to.
- **Q-Learning Control Algorithm:**
- **Q-Learning Algorithm for Off-policy Control:**
- **Q-learning & Exploration Demo:**
 - <https://www.aslanides.io/aixijs/demo.html>

Wrap-up

- **Take home messages:**
 - Model-Free control leverages action-value function.
 - Greedy policy improvement does not need MDP.
 - Generalized policy iteration.
 - Need to maintain sufficient exploration (ϵ -greedy).
 - Off-policy control.
 - Learning value function of a target policy from data generated by a different behaviour policy.
 - Importance sampling to match the expectations of two policies.
 - TD control.
 - On-policy: SARSA(λ).
 - Off-policy: Q-learning.

Next Lecture

- Value-function approximation.
- Leave aside tabular environments.
- Estimate value function with function approximation.
- Linear models & neural networks.
- MC & TD with Stochastic Gradient.

